

```
!pip install -q faiss-cpu transformers accelerate sentencepiece PyPDF2
```

```
===== 23.8/23.8 MB 59.3 MB/s eta 0:00:00
===== 232.6/232.6 kB 22.9 MB/s eta 0:00:00
```

```
import faiss
import numpy as np
import torch
from transformers import AutoTokenizer, AutoModel, AutoModelForCausalLM
from PyPDF2 import PdfReader
```

```
device="cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)
```

```
Using device: cuda
```

```
!pip install -q faiss-cpu transformers accelerate sentencepiece PyPDF2 requests
import faiss
import numpy as np
import torch
import requests
from transformers import AutoTokenizer, AutoModel, AutoModelForCausalLM
from PyPDF2 import PdfReader
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)
```

```
pdf_url = "https://arxiv.org/pdf/1706.03762.pdf"
pdf_path = "paper.pdf"
```

```
response = requests.get(pdf_url)
with open(pdf_path, "wb") as f:
    f.write(response.content)
```

```
print("PDF downloaded!")
```

```
enc_name = "sentence-transformers/all-MiniLM-L6-v2"
```

```
enc_tokenizer = AutoTokenizer.from_pretrained(enc_name)
enc_model = AutoModel.from_pretrained(enc_name).to(device)
enc_model.eval()
```

```
llm_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
```

```
llm_tokenizer = AutoTokenizer.from_pretrained(llm_name)
llm_tokenizer.pad_token = llm_tokenizer.eos_token
```

```
llm_model = AutoModelForCausalLM.from_pretrained(
    llm_name,
```

```

    torch_dtype=torch.float16 if device == "cuda" else torch.float32,
    device_map="auto"
)
llm_model.eval()

def extract_text_from_pdf(path):
    reader = PdfReader(path)
    text = ""
    for page in reader.pages:
        page_text = page.extract_text() or ""
        text += page_text + "\n"
    return text

def chunk_text(text, chunk_size=200, overlap=50):
    words = text.split()
    chunks = []
    start = 0

    while start < len(words):
        end = start + chunk_size
        chunk = " ".join(words[start:end])
        chunks.append(chunk)
        start += (chunk_size - overlap)

    return chunks

def encode_chunks(chunks, batch_size=8):
    all_embeddings = []

    with torch.no_grad():
        for i in range(0, len(chunks), batch_size):
            batch = chunks[i:i+batch_size]

            tokens = enc_tokenizer(
                batch,
                padding=True,
                truncation=True,
                return_tensors="pt"
            ).to(device)

            outputs = enc_model(**tokens)

            embeddings = outputs.last_hidden_state.mean(dim=1)
            all_embeddings.append(
                embeddings.cpu().numpy().astype("float32")
            )

    return np.vstack(all_embeddings)

raw_text = extract_text_from_pdf(pdf_path)

doc_chunks = chunk_text(raw_text)
doc_embeddings = encode_chunks(doc_chunks)

```

```

faiss.normalize_L2(doc_embeddings)

dim = doc_embeddings.shape[1]
index = faiss.IndexFlatIP(dim)
index.add(doc_embeddings)

print("Total chunks indexed:", len(doc_chunks))

def retrieve_context(query, top_k=5):
    with torch.no_grad():
        tokens = enc_tokenizer(
            [query],
            padding=True,
            truncation=True,
            return_tensors="pt"
        ).to(device)

        outputs = enc_model(**tokens)
        q_emb = outputs.last_hidden_state.mean(dim=1)

        q_emb = q_emb.cpu().numpy().astype("float32")
        faiss.normalize_L2(q_emb)

        scores, indices = index.search(q_emb, top_k)

    return [doc_chunks[i] for i in indices[0]]

def generate_rag_answer(question, top_k=5, max_new_tokens=200):
    contexts = retrieve_context(question, top_k)
    context_text = "\n\n".join(contexts)

    prompt = f"""
You are an assistant that answers ONLY using the given context.

Context:
{context_text}

Question: {question}
Answer:
"""

    inputs = llm_tokenizer(
        prompt,
        return_tensors="pt",
        truncation=True,
        max_length=1024
    ).to(device)

    with torch.no_grad():
        output_ids = llm_model.generate(
            *inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False
        )

    output_text = llm_tokenizer.decode(

```

```

        output_ids[0],
        skip_special_tokens=True
    )

```

```

    return output_text.split("Answer:")[1].strip()

```

```

question = "What is the main contribution of this paper?"
answer = generate_rag_answer(question)

```

```

print("\nQuestion:", question)
print("\nAnswer:", answer)

```

Using device: cuda
PDF downloaded!

Loading weights: 100% 103/103 [00:00<00:00, 489.31it/s, Materializing param=pooler.dense.weight]

BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2

Key	Status	
embeddings.position_ids	UNEXPECTED	

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

Loading weights: 100% 201/201 [00:03<00:00, 77.68it/s, Materializing param=model.norm.weight]

Total chunks indexed: 40

Question: What is the main contribution of this paper?

Answer: You are an assistant that answers ONLY using the given context.

Context:

and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. Computational linguistics , 19(2):313-330, 1993. [26] David McClosky, Eugene Charniak,

pages 434-443. ACL, August 2013. 12 Attention Visualizations Input-Input Layer5 It is in this spirit that a majority of American governments have passed new laws since 2009 ma

Recognition , pages 770-778, 2016. [12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning lon

